Exercícios de classes em JavaScript

Classe, constructor, this, throw, try/catch e for...of | 5 exercícios em 1 hora

Ex.	Tema	Tempo
1	Livro: validação no constructor	10 min
2	Playlist: cada objeto com a sua lista	10 min
3	Carrinho: for...of e total	15 min
4	Alunos: try/catch dentro do laço	15 min
5	Caça-erros e perguntas	10 min

Como fazer

- Resolva sozinho, um exercício por vez. Antes de rodar, **escreva a previsão** do que vai aparecer.
- Depois de rodar, compare a sua saída com a esperada **caractere por caractere**: maiúsculas, pontos finais e espaços contam.
- Se travar por mais de 5 minutos, pare e anote: qual linha, e o que você achou que ela fazia. Essa anotação é o que eu preciso para te ajudar.
- Mensagens de erro devem ser idênticas às do enunciado.

Exercício 1: Livro

10 minutos | constructor, validação, campo com valor inicial

Crie a classe **Livro** com os atributos `titulo`, `paginas` e `lido`.

- `constructor(titulo, paginas)`: valide nesta ordem. Título obrigatório, sem espaços nas pontas (mensagem: `Título obrigatório.`). Páginas deve ser inteiro maior ou igual a 1 (mensagem: `Número de páginas inválido.`).
- `lido` começa como `false`, como campo da classe.
- `marcarComoLido()`: apenas muda o estado, **sem retornar nada**.
- `descrever()`: **retorna** o texto no formato da saída (não imprime).

Teste: crie `' Dom Casmurro '` com 256 páginas e mostre a descrição. Chame `marcarComoLido()` e mostre de novo. Depois, dentro de `try/catch`, tente criar um livro com 0 páginas e mostre `Erro:` seguido da mensagem.

SAÍDA ESPERADA

```
Dom Casmurro | 256 páginas | não lido
Dom Casmurro | 256 páginas | lido
Erro: Número de páginas inválido.
```

Exercício 2: Playlist

10 minutos | cada objeto tem os seus próprios dados

Crie a classe **Playlist** com `nome` e `musicas` (uma lista).

- `constructor(nome)`: nome obrigatório, sem espaços nas pontas (`Nome da playlist obrigatório.`). A lista `musicas` nasce vazia.
- `adicionar(musica)`: valida que a música não é vazia (`Música obrigatória.`) e guarda com `push`, sem espaços nas pontas.
- `tamanho()`: retorna quantas músicas existem.

Teste: crie as playlists `Estudo` e `Treino`. Adicione duas músicas em Estudo e uma em Treino. Mostre o tamanho de cada uma. Por último, dentro de `try/catch`, tente adicionar `' '` em Treino.

SAÍDA ESPERADA

```
Estudo: 2 música(s)
Treino: 1 música(s)
Erro: Música obrigatória.
```

Pergunta (responda por escrito): a lista é criada dentro do constructor. O que aconteceria com a playlist Treino se as duas playlists usassem a **mesma** lista criada fora da classe?

Exercício 3: Carrinho de compras

15 minutos | objetos dentro de lista, for...of, acumulador

Crie a classe **Carrinho** com a lista `itens`, vazia ao nascer. Cada item é um objeto `{ nome, preco, quantidade }`.

- `adicionarItem(nome, preco, quantidade)`: valide nesta ordem. Nome obrigatório (`Nome do item obrigatório.`). Preço número finito maior que 0 (`Preço inválido.`). Quantidade inteira maior ou igual a 1 (`Quantidade inválida.`). Se estiver tudo certo, guarde o item com `push`.
- `calcularTotal()`: percorre os itens com **for...of**, soma `preco * quantidade` de cada um e **retorna** o total.

Teste: adicione `caderno` (12.5, 2 unidades), `caneta` (3, 4) e `borracha` (2, 1). Percorra `itens` com for...of mostrando cada linha, depois mostre o total com duas casas decimais. Por fim, dentro de `try/catch`, tente adicionar uma mochila com preço `-10`.

SAÍDA ESPERADA

```
caderno x2 - R$ 25.00
caneta x4 - R$ 12.00
borracha x1 - R$ 2.00
Total: R$ 39.00
Erro: Preço inválido.
```

Atenção: o total é calculado dentro do método, e quem imprime é quem chama. Se o método imprimir, ele devolve `undefined`.

Exercício 4: Cadastro de alunos

15 minutos | try/catch dentro do for...of, contadores

Crie a classe **Aluno** com `nome` e `nota`.

- `constructor(nome, nota)`: nome obrigatório, sem espaços nas pontas (`Nome obrigatório.`). Nota número finito de 0 a 10 (`Nota inválida.`). A nota 0 e a nota 10 são válidas.
- `classificar()`: retorna `Aprovado` se a nota for 7 ou mais, `Recuperação` se for 5 ou mais (e menos que 7) e `Reprovado` nos demais casos.

Copie esta lista para o arquivo, depois da classe:

```
const cadastros = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 6 },
  { nome: " ", nota: 9 },
  { nome: "Carla", nota: 11 },
  { nome: "Davi", nota: 4.5 },
  { nome: "Eva", nota: 7 }
];
```

- Percorra `cadastros` com for...of, numerando a partir de 1. A criação do aluno fica **dentro do try**; a recusa é tratada no `catch`.
- Guarde os alunos aprovados na criação em uma lista `alunos` e, ao final, percorra essa lista contando quantos são Aprovado, Recuperação e Reprovado. Conte também os recusados.

SAÍDA ESPERADA

```
Cadastro 1 ok: Ana - 8.5 - Aprovado
Cadastro 2 ok: Bruno - 6 - Recuperação
Cadastro 3 recusado: Nome obrigatório.
Cadastro 4 recusado: Nota inválida.
```

```
Cadastro 5 ok: Davi - 4.5 - Reprovado
Cadastro 6 ok: Eva - 7 - Aprovado
Aprovados: 2 | Recuperação: 1 | Reprovados: 1 | Recusados: 2
```

Pergunta (responda por escrito): o cadastro 3 deu erro, mas o 4 foi processado. Por quê? O que aconteceria com os cadastros 4 a 6 se a criação do aluno estivesse fora do `try`?

Exercício 5: Caça-erros e perguntas

10 minutos | ler código com atenção

O código abaixo tem **6 erros**, todos de tipos que já apareceram nas suas aulas. Não rode antes de procurar. Para cada erro, escreva três coisas: o **número da linha**, **por que** está errado e **como fica a correção**.

```
01  class Lampada {
02      ligada = false;
03
04      constructor(comodo) {
05          if (comodo.trim() = "") {
06              throw new error("Cômodo obrigatório.");
07          }
08          this.comodo.trim() = comodo;
09      }
10
11      ligar() {
12          return console.log(this.ligada = true);
13      }
14  }
15
16  const sala = Lampada("Sala");
17  sala,ligar();
```

Depois, responda com as suas palavras, em até duas frases cada:

- De quem é o atributo: da classe ou do objeto? Em que momento ele passa a existir?
- O que o `new` faz? Por que sem ele o constructor dá erro?
- Dentro de um método, o que o `this` aponta? Por que escrevemos `this.peso` e não só `peso`?

Checklist de entrega

- Um arquivo por exercício (ou um arquivo só, com os exercícios separados por comentários).
- Saídas do terminal iguais às esperadas.
- Previsões escritas antes de rodar, e respostas por escrito às perguntas.
- Traga o que travar: linha e o que você achou que ela fazia.

Código limpo é civilização.